

SOWMYA.KCODE

DSA DEMYSTIFIED

The Complete Student-Friendly Handbook

Phone-First Multi-Language Master Edition (Java * C++ * Python)

HANDBOOK ARCHITECTURE & LEARNING SYSTEM

- * 5 Core Questions Answered in Every Single Chapter:
(WHAT, WHY, HOW, WHEN, & PATTERN RECOGNITION)
- * Complete Triple-Language Code Labs (Java * C++ * Python).
- * Intuitive Visual Models and Clean ASCII Dry-Runs.
- * Practice Interview Challenges with Hints & Answers.
- * Zero Broken Fonts or Corrupted Characters.
- * 14 Master Coding Patterns & My 90-Day Action Roadmap.

JAVA

Primary Teaching

C++

Fast Execution

PYTHON

Clean Syntax

BUILT SPECIFICALLY FOR:

- * Beginners looking for clear intuition without scary math.
- * College students preparing for campus recruitment tests.
- * Developers gearing up for product-based company rounds.
- * Working professionals switching to software engineering.

OFFICIAL SOWMYA.KCODE VERIFIED RELEASE

Hand-crafted curriculum. Multi-language verified code in Java, C++, Python.

Author: Sowmya Katkojwal | Instagram: @sowmya.kcode
Commercial Release | Price: Rs 39

SYLLABUS : COMPLETE 16-PART CURRICULUM

- Part 0** : Foundations & Big-O Complexity (Zero Math Growth Curves)
- Part 1** : Arrays & Dynamic Arrays (Two Pointers, Move Zeroes)
- Part 2** : Strings & Character Manipulation (26-Bucket Alphabet Array)
- Part 3** : Searching Algorithms (Binary Search 3-Invariant Master Template)
- Part 4** : Sorting Algorithms (Elementary vs Merge Sort Divide & Conquer)
- Part 5** : Linked Lists (Pointer Rewiring, In-Place Reversal, Cycle Detect)
- Part 6** : Stacks (LIFO Cafeteria Plates, Valid Parentheses Matching)
- Part 7** : Queues & Deques (FIFO Line, BFS Level-Order Engine)
- Part 8** : Hashing & HashMaps (O(1) Lookups, Two Sum Key-Value Hashing)
- Part 9** : Recursion & Backtracking (Call Stack, Subsets Power Set)
- Part 10** : Trees & BST (Hierarchies, Maximum Depth, Inorder Traversal)
- Part 11** : Heaps & Priority Queues (Min-Heap, Kth Largest Element)
- Part 12** : Graphs & Networks (Adjacency Lists, Number of Islands DFS)
- Part 13** : Dynamic Programming (Climbing Stairs 1D Tabulation)
- Part 14** : The 14 Master Interview Patterns & Recognition Guide
- Part 15** : My 90-Day Action Roadmap & Live Technical Strategy

THE 5 QUESTIONS MASTER LEARNING FRAMEWORK

Every single chapter in this course strictly answers these five questions:

- 1. WHAT IS THIS?**
Gives a crystal-clear definition in plain, student-friendly language.
- 2. WHY DO WE NEED IT?**
Explains the practical motivation and the real problem it solves.
- 3. HOW DOES IT WORK?**
Details the underlying algorithmic or memory mechanics step-by-step.
- 4. WHEN SHOULD I USE IT?**
Outlines optimal scenarios where this structure or algorithm shines.
- 5. HOW DO I RECOGNIZE IT?**
Provides keywords, hints, and problem patterns to spot it in interviews.

TRIPLE-LANGUAGE CODE LABS INCLUDED (JAVA * C++ * PYTHON)

Each topic contains verified, clean solutions in Java, C++, and Python with syntax notes and complexity.

PART 0 : FOUNDATIONS & BIG-O COMPLEXITY

THE 5 CORE QUESTIONS ANSWERED

- 1. **WHAT:** A mathematical notation describing how algorithm time or memory scales with input size (n).
- 2. **WHY:** Stopwatches measure hardware speed; Big-O measures true algorithm efficiency.
- 3. **HOW:** Count dominant operations, discard constant factors ($O(3n) \rightarrow O(n)$), keep highest power term.
- 4. **WHEN:** Analyze before coding to verify the algorithm fits within interview runtime limits (10^8 ops/sec).
- 5. **RECOGNIZE:** Check constraints: $n \leq 10^5$ demands $O(n)$ or $O(n \log n)$; $n \leq 1000$ allows $O(n^2)$.

BIG-O GROWTH CURVES (ZERO MATH INTUITION)

Operations			
n^2		/	$O(n^2)$ [Slow: Nested Loops]
n		/	$O(n)$ [Fair: Single Loop]
$\log n$			$O(\log n)$ [Very Fast: Halving Space]
1			$O(1)$ [Constant: Direct Memory Jump]
+-----> Input Size (n)			

INTERVIEW CHALLENGE : ARRAY SEARCH COMPARISON

QUESTION: Determine the time complexity of searching for target x in an unsorted array vs a sorted array.

HINT: Can you eliminate half the remaining numbers without checking them individually?

APPROACH & ANSWER: Unsorted array requires Linear Search: $O(n)$. Sorted array allows Binary Search: $O(\log n)$.

EXPLANATION: In an unsorted array, x could be anywhere. In a sorted array, comparing x with mid cuts candidates by 50%.

TIME: $O(\log n)$ sorted vs $O(n)$ unsorted | SPACE: $O(1)$ auxiliary memory

COMMON MISTAKE

Assuming Big-O measures seconds. It measures operations growth as n scales.

PRO INTERVIEW TIP

Drop constants: $O(2n + 50)$ simplifies directly to $O(n)$ in interviews.

PART 1 : ARRAYS & DYNAMIC ARRAYS

THE 5 CORE QUESTIONS ANSWERED

- 1. **WHAT:** Contiguous block of memory holding elements of identical type at indices 0 to n-1.
- 2. **WHY:** Direct memory addressing (base + index * size) enables instant O(1) reads and writes.
- 3. **HOW:** Fixed arrays have locked size. Dynamic arrays (ArrayList) auto-expand 2x when capacity fills.
- 4. **WHEN:** Use for fast index-based lookups, contiguous buffer storage, and sequential scans.
- 5. **RECOGNIZE:** Keywords: "sorted array", "find pair", "subarray", "two pointers", "sliding window".

TWO POINTERS CONVERGING INWARD (TARGET = 9)

```
[ 2,   7,  11,  15 ]   target = 9
  ^               ^
left             right
[ 2,   7,  11,  15 ]   2 + 15 = 17 (> 9) -> sum too large, move right--
  ^               ^
  2 + 7 = 9 (MATCH!) -> return 1-based indices [1, 2]
```

INTERVIEW CHALLENGE : MOVE ZEROES (LEETCODE 283)

QUESTION: Move all 0s to the end of nums while maintaining relative order of non-zero elements.

HINT: Keep a write pointer tracking where the next non-zero element belongs.

APPROACH & ANSWER: Iterate read pointer; when nums[read] != 0, write to nums[write++] then fill trailing with 0.

EXPLANATION: Overwrites non-zeroes forward in one pass. Fills remaining indices with 0 in second pass.

TIME: O(n) single pass | **SPACE:** O(1) in-place modification

COMMON MISTAKE

Creating a new array when in-place modification is required.

PRO INTERVIEW TIP

For sorted pair searching, Two Pointers drops brute force O(n^2) to linear O(n).

PART 1 : ARRAYS : CODE LAB**MULTI-LANGUAGE IMPLEMENTATIONS:** **JAVA** (Primary) | **C++** (STL) | **PYTHON** (Clean)**JAVA : Two Sum II (Sorted Array)**

```
public static int[] twoSum(int[] numbers, int target) {
    int left = 0, right = numbers.length - 1;
    while (left < right) {
        int sum = numbers[left] + numbers[right];
        if (sum == target) return new int[]{left + 1, right + 1};
        else if (sum < target) left++; // Increase sum
        else right--; // Decrease sum
    }
    return new int[]{};
}
```

* Pointers start at opposite ends; sorted property guarantees directional movement.

C++ : Two Sum II (Sorted Array)

```
vector<int> twoSum(const vector<int>& numbers, int target) {
    int left = 0, right = numbers.size() - 1;
    while (left < right) {
        int sum = numbers[left] + numbers[right];
        if (sum == target) return {left + 1, right + 1};
        else if (sum < target) left++;
        else right--;
    }
    return {};
}
```

* Pass vector by const reference to avoid expensive deep copies.

PYTHON : Two Sum II (Sorted Array)

```
def two_sum(numbers: list[int], target: int) -> list[int]:
    left, right = 0, len(numbers) - 1
    while left < right:
        curr_sum = numbers[left] + numbers[right]
        if curr_sum == target: return [left + 1, right + 1]
        elif curr_sum < target: left += 1
        else: right -= 1
    return []
```

* Python list indexing is 0-based; problem requests 1-based index return.

SHARED COMPLEXITY: TIME $O(n)$ Single Pass | SPACE $O(1)$ Auxiliary Memory

Key Takeaway: Two pointers converging inward drops brute-force $O(n^2)$ to clean linear $O(n)$.

PART 2 : STRINGS & CHARACTER MANIPULATION

THE 5 CORE QUESTIONS ANSWERED

- 1. **WHAT:** Ordered sequence of characters. In Java and Python, strings are completely immutable.
- 2. **WHY:** Immutable strings prevent accidental corruption but repeated `str += "a"` causes $O(n^2)$ copies.
- 3. **HOW:** Use `StringBuilder` for $O(1)$ appends. Use 26-element integer array for letter frequency counting.
- 4. **WHEN:** Use frequency buckets when character set is bounded (lowercase a-z) for $O(1)$ auxiliary space.
- 5. **RECOGNIZE:** Keywords: "anagram", "palindrome", "substring", "character frequency", "ASCII diff".

THE 26-BUCKET ALPHABET ARRAY (INDEX = CHAR - 'A')

Char: 'a' 'b' 'c' ... 'z'
Index: 0 1 2 ... 25
Math: 'c' - 'a' = 99 - 97 = 2 (Instant $O(1)$ bucket mapping!)

INTERVIEW CHALLENGE : VALID PALINDROME (LEETCODE 125)

QUESTION: Check if string reads same backwards after converting to lowercase and stripping non-alphanumeric.

HINT: Two pointers converging inward; skip non-alphanumeric via `Character.isLetterOrDigit()`.

APPROACH & ANSWER: `left = 0, right = s.length() - 1`. Compare chars; if mismatch return false.

EXPLANATION: Pointers advance inward, skipping punctuation. Returns true if pointers meet without mismatch.

TIME: $O(n)$ linear scan | **SPACE:** $O(1)$ in-place pointers

COMMON MISTAKE

Comparing Java strings with `==` (compares memory addresses!). Always use `.equals()`.

PRO INTERVIEW TIP

A 26-slot integer array beats `HashMap` in speed and memory for lowercase alphabet.

PART 2 : STRINGS : CODE LAB**MULTI-LANGUAGE IMPLEMENTATIONS:** **JAVA** (Primary) | **C++** (STL) | **PYTHON** (Clean)**JAVA : Valid Anagram (LeetCode 242)**

```
public static boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < s.length(); i++) {
        count[s.charAt(i) - 'a']++;
        count[t.charAt(i) - 'a']--;
    }
    for (int c : count) if (c != 0) return false;
    return true;
}
```

* Single loop increments bucket for s and decrements for t simultaneously.

C++ : Valid Anagram (LeetCode 242)

```
bool isAnagram(const string& s, const string& t) {
    if (s.length() != t.length()) return false;
    vector<int> count(26, 0);
    for (int i = 0; i < s.length(); i++) {
        count[s[i] - 'a']++;
        count[t[i] - 'a']--;
    }
    for (int c : count) if (c != 0) return false;
    return true;
}
```

* Fixed vector of size 26 avoids hashing overhead of std::unordered_map.

PYTHON : Valid Anagram (LeetCode 242)

```
def is_anagram(s: str, t: str) -> bool:
    if len(s) != len(t): return False
    count = [0] * 26
    for ch_s, ch_t in zip(s, t):
        count[ord(ch_s) - ord('a')] += 1
        count[ord(ch_t) - ord('a')] -= 1
    return all(c == 0 for c in count)
```

* ord() converts character to ASCII code; zip() pairs characters cleanly.

SHARED COMPLEXITY: TIME $O(n)$ Linear Scan | SPACE $O(1)$ Fixed 26-Slot Array

Key Takeaway: For fixed alphabets, a 26-element array is 10x faster than HashMap lookups.

PART 3 : SEARCHING ALGORITHMS

THE 5 CORE QUESTIONS ANSWERED

- 1. WHAT:** Algorithms to locate target elements. Linear search checks all; Binary Search halves candidates.
- 2. WHY:** Linear search takes 1,000,000 checks for $n = 1,000,000$. Binary search takes only 20 checks!
- 3. HOW:** Compare target with midpoint. If target is smaller, search left half; else search right half.
- 4. WHEN:** Use Binary Search when array is sorted, or when search space has monotonic true/false condition.
- 5. RECOGNIZE:** Keywords: "sorted array", "find minimum in rotated", " $O(\log n)$ time requirement".

BINARY SEARCH INVARIANT (TARGET = 23)

Left=0

Mid=3

Right=6

[2, 5, 8, |12|, 16, 23, 38]

Target 23 > Mid 12 -> Eliminate left half -> left = mid + 1 (Index 4)

INTERVIEW CHALLENGE : SEARCH IN ROTATED SORTED ARRAY (LEETCODE 33)

QUESTION: Search target in sorted array rotated at unknown pivot. Must achieve $O(\log n)$ runtime.

HINT: At least one half (left..mid or mid..right) is always strictly sorted!

APPROACH & ANSWER: Check which half is sorted. If target lies within sorted half bounds, search it; else search other half.

EXPLANATION: Determining the sorted half allows discarding 50% candidates in every iteration.

TIME: $O(\log n)$ logarithmic | **SPACE:** $O(1)$ constant auxiliary space

COMMON MISTAKE

Using $(\text{left} + \text{right}) / 2$ causes integer overflow when $\text{left} + \text{right} > 2^{31} - 1$.

PRO INTERVIEW TIP

Always use $\text{left} + (\text{right} - \text{left}) / 2$ to safely calculate the midpoint.

PART 3 : SEARCHING : CODE LAB**MULTI-LANGUAGE IMPLEMENTATIONS:** [JAVA \(Primary\)](#) | [C++ \(STL\)](#) | [PYTHON \(Clean\)](#)**JAVA : Binary Search (LeetCode 704)**

```
public static int binarySearch(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // Safe from overflow
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

* Condition `left <= right` ensures single remaining element is properly evaluated.

C++ : Binary Search (LeetCode 704)

```
int binarySearch(const vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

* Standard STL provides `std::lower_bound` and `std::binary_search`.

PYTHON : Binary Search (LeetCode 704)

```
def binary_search(nums: list[int], target: int) -> int:
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if nums[mid] == target: return mid
        elif nums[mid] < target: left = mid + 1
        else: right = mid - 1
    return -1
```

* `//` operator performs integer floor division in Python 3.

SHARED COMPLEXITY: TIME $O(\log n)$ Logarithmic | SPACE $O(1)$ Constant Auxiliary

Key Takeaway: Halving the search space every iteration cuts 1,000,000 items in 20 comparisons.

PART 4 : SORTING ALGORITHMS

THE 5 CORE QUESTIONS ANSWERED

- 1. WHAT:** Arranging elements into ascending or descending numerical / alphabetical order.
- 2. WHY:** Sorted data unlocks $O(\log n)$ Binary Search, $O(n)$ Two Pointers, and eliminates duplicate checks.
- 3. HOW:** Elementary sorts (Bubble, Selection) take $O(n^2)$. Divide-and-conquer (Merge Sort) achieves $O(n \log n)$.
- 4. WHEN:** Sort before processing when order allows greedy choices or fast interval comparisons.
- 5. RECOGNIZE:** Keywords: "kth smallest", "merge intervals", "meeting rooms", "sort colors".

MERGE SORT DIVIDE & CONQUER TREE

Split: [38, 27, 43, 3, 9, 82, 10]
Divide: [38, 27, 43] [3, 9, 82, 10]
Base: [38] [27] [43] [3] [9] [82] [10]
Merge: [27, 38, 43] [3, 9, 10, 82]
Final: [3, 9, 10, 27, 38, 43, 82]

INTERVIEW CHALLENGE : QUICK SORT PIVOT PARTITIONING

QUESTION: Explain why Quick Sort average case is $O(n \log n)$ but worst case degrades to $O(n^2)$.

HINT: What happens when array is already sorted and you pick the last element as pivot?

APPROACH & ANSWER: Balanced pivot divides array in half (height $\log n$). Unbalanced pivot divides into 1 and $n-1$ (height n).

EXPLANATION: Use randomized pivot selection to ensure average $O(n \log n)$ runtime in real interviews.

TIME: $O(n \log n)$ average, $O(n^2)$ worst | **SPACE:** $O(\log n)$ call stack

COMMON MISTAKE

Using Bubble Sort in coding interviews. Interviewers expect $O(n \log n)$ mastery.

PRO INTERVIEW TIP

Merge Sort is stable and guarantees $O(n \log n)$ even in the worst case scenario.

PART 4 : SORTING : CODE LAB**MULTI-LANGUAGE IMPLEMENTATIONS:** **JAVA** (Primary) | **C++** (STL) | **PYTHON** (Clean)**JAVA : Merge Sort (Divide & Conquer)**

```
public static void mergeSort(int[] arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right); // Merges two sorted halves
}
```

* Recursively divides array into singletons then merges them back in sorted order.

C++ : Merge Sort (Divide & Conquer)

```
void mergeSort(vector<int>& arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}
```

* In-place std::sort uses Introsort (QuickSort + HeapSort hybrid) taking $O(n \log n)$.

PYTHON : Merge Sort (Divide & Conquer)

```
def merge_sort(arr: list[int]) -> list[int]:
    if len(arr) <= 1: return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
```

* Python built-in Timsort (.sort() / sorted()) combines Merge and Insertion sort.

SHARED COMPLEXITY: TIME $O(n \log n)$ Guaranteed | SPACE $O(n)$ Auxiliary Buffer

Key Takeaway: Divide and conquer breaks problem into $\log n$ levels with n work per level.

PART 5 : LINKED LISTS

THE 5 CORE QUESTIONS ANSWERED

- 1. **WHAT:** Linear data structure where elements (nodes) contain data and pointer to next node.
- 2. **WHY:** Dynamic size with O(1) head insertion/deletion without shifting elements in memory.
- 3. **HOW:** Nodes scattered across heap memory; sequential pointer traversal replaces index math.
- 4. **WHEN:** Use when constant-time insertion/deletion at boundaries is needed (LRU cache, queues).
- 5. **RECOGNIZE:** Keywords: "reverse list", "middle of list", "cycle detection", "dummy node".

IN-PLACE POINTER REWIRING (REVERSE LINKED LIST)

Before: [1] ---> [2] ---> [3] ---> null
Step: next = curr.next; curr.next = prev; prev = curr; curr = next;
After: null <--- [1] <--- [2] <--- [3] (prev is new head!)

INTERVIEW CHALLENGE : LINKED LIST CYCLE (LEETCODE 141)

QUESTION: Determine if a linked list contains a cycle using O(1) auxiliary memory.

HINT: Floyd's Tortoise and Hare algorithm: slow moves 1 step, fast moves 2 steps.

APPROACH & ANSWER: If slow == fast at any point, cycle exists. If fast or fast.next reaches null, no cycle.

EXPLANATION: Inside a cycle of length C, the gap between fast and slow shrinks by 1 step every iteration.

TIME: O(n) linear traversal | SPACE: O(1) two pointers

COMMON MISTAKE

Losing track of next node before rewiring pointer (causes memory orphan leak).

PRO INTERVIEW TIP

Always save next = curr.next before changing curr.next to prev!

PART 5 : LINKED LISTS : CODE LAB**MULTI-LANGUAGE IMPLEMENTATIONS:** **JAVA** (Primary) | **C++** (STL) | **PYTHON** (Clean)**JAVA : Reverse Linked List (LeetCode 206)**

```
public static ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next; // 1. Save next
        curr.next = prev;          // 2. Reverse pointer
        prev = curr;               // 3. Advance prev
        curr = next;               // 4. Advance curr
    }
    return prev; // New head
}
```

* 4-step pointer dance rewires links in-place without allocating new nodes.

C++ : Reverse Linked List (LeetCode 206)

```
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr != nullptr) {
        ListNode* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

* Use nullptr in modern C++ instead of NULL for type-safe pointer initialization.

PYTHON : Reverse Linked List (LeetCode 206)

```
def reverse_list(head: Optional[ListNode]) -> Optional[ListNode]:
    prev, curr = None, head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev
```

* Python multiple assignment allows: curr.next, prev, curr = prev, curr, curr.next

SHARED COMPLEXITY: TIME $O(n)$ Single Pass | SPACE $O(1)$ In-Place Rewiring

Key Takeaway: Linked list manipulation is all about careful pointer temporary storage.

PART 6 : STACKS

THE 5 CORE QUESTIONS ANSWERED

- 1. **WHAT:** LIFO (Last-In, First-Out) linear structure. Elements added and removed only at top.
- 2. **WHY:** Tracks execution context, nested syntax hierarchies, and reverses sequence naturally.
- 3. **HOW:** push() adds to top in O(1); pop() removes from top in O(1); peek() views top in O(1).
- 4. **WHEN:** Use for matching brackets, undo/redo buffers, evaluating expressions, and monotonic scans.
- 5. **RECOGNIZE:** Keywords: "valid parentheses", "next greater element", "monotonic stack", "undo".

LIFO STACK MECHANISM & BRACKET MATCHING

Push: push('(') -> ['(']
Match: Incoming ')' matches top '(' -> pop() -> Stack empty (VALID!)
Rule: Every closing bracket must match the most recently opened bracket.

INTERVIEW CHALLENGE : DAILY TEMPERATURES (LEETCODE 739)

QUESTION: Given daily temperatures, return array showing days to wait until a warmer temperature.

HINT: Maintain a Monotonic Decreasing Stack storing indices of unresolved temperatures.

APPROACH & ANSWER: When today's temp > stack.peek(), pop index and record difference: result[prev] = today - prev.

EXPLANATION: Every index is pushed and popped at most once, yielding linear O(n) total runtime.

TIME: O(n) single pass | SPACE: O(n) stack memory

COMMON MISTAKE

Using legacy Stack in Java (thread-safe, slow). Use ArrayDeque for 3x speed.

PRO INTERVIEW TIP

Always check !stack.isEmpty() before calling stack.pop() to prevent empty stack exception.

PART 6 : STACKS : CODE LAB**MULTI-LANGUAGE IMPLEMENTATIONS:** **JAVA** (Primary) | **C++** (STL) | **PYTHON** (Clean)**JAVA : Valid Parentheses (LeetCode 20)**

```
public static boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push('(');
        else if (c == '{') stack.push('{');
        else if (c == '[') stack.push('[');
        else if (stack.isEmpty() || stack.pop() != c) return false;
    }
    return stack.isEmpty();
}
```

* Push expected closing bracket; compare directly on encountering close character.

C++ : Valid Parentheses (LeetCode 20)

```
bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(') st.push('(');
        else if (c == '{') st.push('{');
        else if (c == '[') st.push('[');
        else {
            if (st.empty() || st.top() != c) return false;
            st.pop();
        }
    }
    return st.empty();
}
```

* C++ std::stack::top() inspects value; std::stack::pop() removes it (void return).

PYTHON : Valid Parentheses (LeetCode 20)

```
def is_valid(s: str) -> bool:
    stack = []
    pairs = {'(': ')', '{': '}', '[': ']'}
    for ch in s:
        if ch in pairs: stack.append(pairs[ch])
        elif not stack or stack.pop() != ch: return False
    return len(stack) == 0
```

* Standard Python list serves as an optimal stack using append() and pop().

SHARED COMPLEXITY: TIME $O(n)$ Single Pass | SPACE $O(n)$ Stack Storage

Key Takeaway: Pushing expected closing character simplifies matching logic to a single check.

PART 7 : QUEUES & DEQUES

THE 5 CORE QUESTIONS ANSWERED

- 1. **WHAT:** FIFO (First-In, First-Out) structure. Deque supports O(1) push/pop at both front and back.
- 2. **WHY:** Guarantees fair processing in order of arrival; forms the core engine for BFS graph traversal.
- 3. **HOW:** offer() adds to back; poll() removes from front; peek() inspects front element.
- 4. **WHEN:** Use for level-order tree traversal, shortest path in unweighted graphs, and printer buffers.
- 5. **RECOGNIZE:** Keywords: "level order", "shortest path", "sliding window maximum", "BFS".

FIFO QUEUE & LEVEL-ORDER BFS WAVEFRONT

Enq: [Root] -> Poll Root -> Enq Left, Right
Level 0: [1]
Level 1: [2, 3]
Level 2: [4, 5, 6, 7] (Natural level-by-level wavefront!)

INTERVIEW CHALLENGE : SLIDING WINDOW MAXIMUM (LEETCODE 239)

QUESTION: Find maximum element in sliding window of size k moving across array nums.

HINT: Maintain a Monotonic Deque storing indices of elements in strictly decreasing order.

APPROACH & ANSWER: Front of deque always holds maximum of current window. Pop smaller elements from back before inserting.

EXPLANATION: Each element enters and exits deque at most once, yielding linear O(n) total runtime.

TIME: O(n) linear scan | SPACE: O(k) window deque

COMMON MISTAKE

Using LinkedList for queue when high performance is critical. ArrayDeque is faster.

PRO INTERVIEW TIP

Record int size = q.size() before level loop to process exactly one tree depth at a time.

PART 7 : QUEUES & BFS : CODE LAB

MULTI-LANGUAGE IMPLEMENTATIONS: **JAVA** (Primary) | **C++** (STL) | **PYTHON** (Clean)**JAVA : Tree Level Order Traversal (LeetCode 102)**

```

public static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size(); // Freeze count for current level
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            level.add(node.val);
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
        res.add(level);
    }
    return res;
}

```

* Freezing int size = q.size() isolates exact elements belonging to current depth.

C++ : Tree Level Order Traversal (LeetCode 102)

```

vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res; if (!root) return res;
    queue<TreeNode*> q; q.push(root);
    while (!q.empty()) {
        int size = q.size(); vector<int> level;
        for (int i = 0; i < size; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}

```

* std::queue::front() retrieves item; std::queue::pop() removes it.

PYTHON : Tree Level Order Traversal (LeetCode 102)

```

def level_order(root: Optional[TreeNode]) -> list[list[int]]:
    if not root: return []
    res, q = [], collections.deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)
    return res

```

* collections.deque provides O(1) popleft(); avoid list.pop(0) which is O(n).

SHARED COMPLEXITY: TIME $O(n)$ Visits Every Node Once | SPACE $O(w)$ Max Tree Width

Key Takeaway: Freezing queue size before inner loop is the universal level-by-level BFS pattern.

PART 8 : HASHING & HASHMAPS

THE 5 CORE QUESTIONS ANSWERED

- 1. WHAT:** Data structure mapping unique keys to values using hash function for $O(1)$ lookups.
- 2. WHY:** Replaces expensive $O(n)$ array scans with instantaneous $O(1)$ key existence checks.
- 3. HOW:** Hash code computes bucket index. Collisions resolved via linked chains or open addressing.
- 4. WHEN:** Use when tracking counts, finding complements, detecting duplicates, or memoization.
- 5. RECOGNIZE:** Keywords: "two sum", "group anagrams", "frequency map", " $O(1)$ lookup", "unique count".

HASHMAP COMPLEMENT LOOKUP (TARGET = 9)

Index 0: val = 2 -> need 9 - 2 = 7 -> not in map -> map.put(2, 0)

Index 1: val = 7 -> need 9 - 7 = 2 -> FOUND IN MAP at index 0!

Return indices [0, 1] in $O(1)$ lookup!

INTERVIEW CHALLENGE : GROUP ANAGRAMS (LEETCODE 49)

QUESTION: Group array of strings into sublists where each sublist contains anagrams of each other.

HINT: Sorted string or 26-character frequency count serves as identical hash key for all anagrams!

APPROACH & ANSWER: Iterate strings. Compute frequency key (e.g. "#1#0#0..."). Store in Map<String, List<String>>.

EXPLANATION: Strings with identical character counts map to identical key bucket in $O(n * k)$ time.

TIME: $O(n * k)$ where k is string length | **SPACE:** $O(n * k)$ map storage

COMMON MISTAKE

Using mutable objects as HashMap keys (mutating key changes hash code, making value lost!).

PRO INTERVIEW TIP

For unsorted arrays, HashMap drops Two Sum brute force $O(n^2)$ to linear $O(n)$.

PART 8 : HASHMAPS : CODE LAB**MULTI-LANGUAGE IMPLEMENTATIONS:** **JAVA** (Primary) | **C++** (STL) | **PYTHON** (Clean)**JAVA : Two Sum (LeetCode 1)**

```
public static int[] twoSumHash(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (map.containsKey(complement)) {
            return new int[]{map.get(complement), i};
        }
        map.put(nums[i], i);
    }
    return new int[]{};
}
```

* Stores (value -> index). Checks complement existence in single linear pass.

C++ : Two Sum (LeetCode 1)

```
vector<int> twoSum(const vector<int>& nums, int target) {
    unordered_map<int, int> seen;
    for (int i = 0; i < nums.size(); i++) {
        int complement = target - nums[i];
        if (seen.count(complement)) {
            return {seen[complement], i};
        }
        seen[nums[i]] = i;
    }
    return {};
```

* Use std::unordered_map for O(1) average hash lookups instead of std::map O(log n).

PYTHON : Two Sum (LeetCode 1)

```
def two_sum(nums: list[int], target: int) -> list[int]:
    seen = {}
    for i, num in enumerate(nums):
        diff = target - num
        if diff in seen:
            return [seen[diff], i]
        seen[num] = i
    return []
```

* Python dict provides O(1) amortized lookup via highly optimized hash table.

SHARED COMPLEXITY: TIME O(n) Single Pass | SPACE O(n) Hash Table

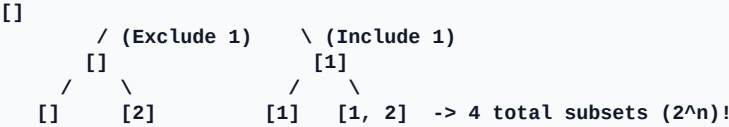
Key Takeaway: Trading auxiliary memory for instantaneous O(1) lookups is DSA's master trade-off.

PART 9 : RECURSION & BACKTRACKING

THE 5 CORE QUESTIONS ANSWERED

- 1. WHAT:** Technique where function calls itself. Backtracking explores candidates, undoing dead ends.
- 2. WHY:** Solves complex combinatorial problems (subsets, permutations, N-Queens) with clean code.
- 3. HOW:** 1. Base Case (stops recursion). 2. Choose candidate. 3. Recurse. 4. Un-choose (backtrack).
- 4. WHEN:** Use when problem demands generating all possible configurations, combinations, or paths.
- 5. RECOGNIZE:** Keywords: "generate all", "subsets", "permutations", "N-Queens", "solve sudoku".

BACKTRACKING DECISION TREE (SUBSETS OF [1, 2])



INTERVIEW CHALLENGE : PERMUTATIONS (LEETCODE 46)

QUESTION: Given an array nums of distinct integers, return all possible permutations.

HINT: Track visited elements using boolean array. Recurse until path length equals nums.length.

APPROACH & ANSWER: Loop over elements. If not visited: mark visited, append to path, recurse, pop, mark unvisited.

EXPLANATION: Systematically explores all n! arrangements, backtracking at each branch.

TIME: O(n * n!) factorial | SPACE: O(n) recursion call stack

COMMON MISTAKE

Forgetting the base case (causes stack overflow exception: java.lang.StackOverflowError).

PRO INTERVIEW TIP

Always undo the state change (e.g. list.remove(list.size() - 1)) during backtracking!

PART 9 : RECURSION & BACKTRACKING : CODE LAB

MULTI-LANGUAGE IMPLEMENTATIONS: JAVA (Primary) | C++ (STL) | PYTHON (Clean)

JAVA : Subsets / Power Set (LeetCode 78)

```
public static List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    backtrack(0, nums, new ArrayList<>(), res);
    return res;
}

private static void backtrack(int start, int[] nums, List<Integer> cur,
    List<List<Integer>> res, ArrayList<Integer> cur) { // Copy snapshot
    for (int i = start; i < nums.length; i++) {
        cur.add(nums[i]); // Choose
        backtrack(i + 1, nums, cur, res); // Explore
        cur.remove(cur.size() - 1); // Un-choose (backtrack)
    }
}
```

* Always pass new ArrayList<>(cur) when saving; otherwise reference gets cleared.

C++ : Subsets / Power Set (LeetCode 78)

```
void backtrack(int start, const vector<int>& nums, vector<int>& cur,
    vector<vector<int>>& res) {
    for (int i = start; i < nums.size(); i++) {
        cur.push_back(nums[i]);
        backtrack(i + 1, nums, cur, res);
        cur.pop_back(); // Backtrack
    }
}

vector<vector<int>> subsets(const vector<int>& nums) {
    vector<vector<int>> res; vector<int> cur;
    backtrack(0, nums, cur, res); return res;
}
```

* std::vector push_back and pop_back naturally implement state transition.

PYTHON : Subsets / Power Set (LeetCode 78)

```
def subsets(nums: list[int]) -> list[list[int]]:
    res = []
    def backtrack(start: int, cur: list[int]):
        res.append(list(cur)) # Copy snapshot
        for i in range(start, len(nums)):
            cur.append(nums[i])
            backtrack(i + 1, cur)
            cur.pop() # Backtrack
    backtrack(0, [])
    return res
```

* list(cur) creates independent shallow copy of candidate list.

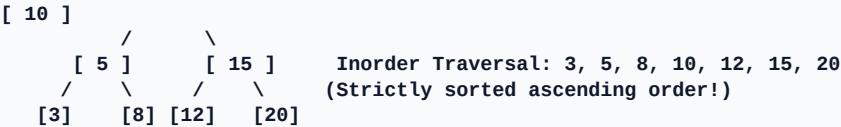
SHARED COMPLEXITY: TIME $O(n * 2^n)$ Combinatorial | SPACE $O(n)$ Recursion Call Stack
Key Takeaway: Choose -> Explore -> Backtrack is the immutable formula for all generation tasks.

PART 10 : TREES & BINARY SEARCH TREES

THE 5 CORE QUESTIONS ANSWERED

- 1. **WHAT:** Hierarchical node structure with root node and subtrees. BST enforces left < root < right.
- 2. **WHY:** Combines fast O(1) flexibility of linked lists with O(log n) search capability of arrays.
- 3. **HOW:** Traversals: Inorder (Left, Root, Right -> sorted in BST), Preorder, Postorder.
- 4. **WHEN:** Use for hierarchical file systems, syntax parsing trees, and sorted dynamic data.
- 5. **RECOGNIZE:** Keywords: "binary search tree", "inorder traversal", "maximum depth", "LCA".

BINARY SEARCH TREE INVARIANT (LEFT < ROOT < RIGHT)



INTERVIEW CHALLENGE : VALIDATE BINARY SEARCH TREE (LEETCODE 98)

QUESTION: Determine if binary tree is valid BST. Left subtree < node.val < Right subtree everywhere.

HINT: Pass valid allowable range [min, max] down the recursion tree.

APPROACH & ANSWER: validate(node, min, max). If node == null return true. If node.val <= min || >= max return false.

EXPLANATION: Ensures every node obeys constraints set by all ancestors, not just direct parent.

TIME: O(n) visits every node once | SPACE: O(h) tree height stack memory

COMMON MISTAKE

Checking only immediate children (left < root and right > root is NOT enough for global BST!).

PRO INTERVIEW TIP

Inorder traversal of a valid BST always produces strictly sorted ascending elements.

PART 10 : TREES : CODE LAB**MULTI-LANGUAGE IMPLEMENTATIONS:** **JAVA** (Primary) | **C++** (STL) | **PYTHON** (Clean)**JAVA : Maximum Depth of Binary Tree (LeetCode 104)**

```
public static int maxDepth(TreeNode root) {  
    if (root == null) return 0;  
    int leftDepth = maxDepth(root.left);  
    int rightDepth = maxDepth(root.right);  
    return 1 + Math.max(leftDepth, rightDepth);  
}
```

* Base case returns 0 for null. Height equals 1 plus max of children subtrees.

C++ : Maximum Depth of Binary Tree (LeetCode 104)

```
int maxDepth(TreeNode* root) {  
    if (!root) return 0;  
    return 1 + max(maxDepth(root->left), maxDepth(root->right));  
}
```

* Concise recursive formulation leverages call stack naturally.

PYTHON : Maximum Depth of Binary Tree (LeetCode 104)

```
def max_depth(root: Optional[TreeNode]) -> int:  
    if not root: return 0  
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

* Python elegant single-line recursion handles null checks gracefully.

SHARED COMPLEXITY: TIME $O(n)$ Node Count | SPACE $O(h)$ Tree Height Call Stack

Key Takeaway: Divide and conquer on trees: solve left child, solve right child, combine at root.

PART 11 : HEAPS & PRIORITY QUEUES

THE 5 CORE QUESTIONS ANSWERED

- 1. WHAT:** Complete binary tree where root is minimum (Min-Heap) or maximum (Max-Heap).
- 2. WHY:** Finds and extracts minimum/maximum element in $O(1)$ peek and $O(\log n)$ removal.
- 3. HOW:** Stored in compact array: parent = $(i - 1) / 2$; left = $2i + 1$; right = $2i + 2$.
- 4. WHEN:** Use for Top-K problems, running stream median, and Dijkstra's shortest path algorithm.
- 5. RECOGNIZE:** Keywords: "kth largest", "top k frequent", "merge k sorted lists", "running median".

MIN-HEAP OF SIZE K = 3 (FINDING 3RD LARGEST)

Stream: [10, 20, 5, 80, 50] k = 3
Insert 10, 20, 5 -> Heap: [5, 10, 20]
Insert 80 -> size 4 -> poll min (5) -> Heap: [10, 20, 80]
Insert 50 -> size 4 -> poll min (10) -> Heap: [20, 50, 80] (Top = 20!)

INTERVIEW CHALLENGE : TOP K FREQUENT ELEMENTS (LEETCODE 347)

QUESTION: Given an integer array nums and integer k, return the k most frequent elements.

HINT: Count frequencies with HashMap, then use Min-Heap of size k comparing frequency counts.

APPROACH & ANSWER: Store entries in Min-Heap based on frequency. When heap size > k, poll. Remaining k are top elements.

EXPLANATION: Maintaining size k heap bounds runtime to $O(n \log k)$, far faster than sorting whole array.

TIME: $O(n \log k)$ bounded heap | **SPACE:** $O(n)$ hash map storage

COMMON MISTAKE

Sorting entire array taking $O(n \log n)$ when only Top-K elements are requested.

PRO INTERVIEW TIP

A Min-Heap of size k finds Kth largest; a Max-Heap of size k finds Kth smallest!

PART 11 : HEAPS : CODE LAB**MULTI-LANGUAGE IMPLEMENTATIONS:** **JAVA** (Primary) | **C++** (STL) | **PYTHON** (Clean)**JAVA : Kth Largest Element (LeetCode 215)**

```
public static int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) minHeap.poll();
    }
    return minHeap.peek();
}
```

* Java PriorityQueue is Min-Heap by default. Top of heap is kth largest.

C++ : Kth Largest Element (LeetCode 215)

```
int findKthLargest(const vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap;
    for (int num : nums) {
        minHeap.push(num);
        if (minHeap.size() > k) minHeap.pop();
    }
    return minHeap.top();
}
```

* Use greater<int> comparator in C++ std::priority_queue to make it a Min-Heap.

PYTHON : Kth Largest Element (LeetCode 215)

```
def find_kth_largest(nums: list[int], k: int) -> int:
    min_heap = []
    for num in nums:
        heapq.heappush(min_heap, num)
        if len(min_heap) > k:
            heapq.heappop(min_heap)
    return min_heap[0]
```

* Python heapq module turns standard list into a Min-Heap in-place.

SHARED COMPLEXITY: TIME $O(n \log k)$ Fast Bounded | SPACE $O(k)$ Bounded Heap Size

Key Takeaway: Maintaining heap size at k evicts smaller candidates and guarantees $\log k$ operations.

PART 12 : GRAPHS & NETWORKS

THE 5 CORE QUESTIONS ANSWERED

- 1. **WHAT:** Network of vertices (nodes) connected by edges. Can be directed/undirected, weighted/unweighted.
- 2. **WHY:** Models real-world relationships: social networks, road routes, dependency trees, internet web.
- 3. **HOW:** Represent via Adjacency List Map<Node, List<Node>>. Traverse via BFS (Queue) or DFS (Stack/Recursion).
- 4. **WHEN:** Use BFS for shortest path in unweighted graphs; use DFS for connectivity, cycles, and islands.
- 5. **RECOGNIZE:** Keywords: "number of islands", "course schedule", "shortest path", "connected components".

GRID GRAPH MODELING (4-DIRECTIONAL DFS)

Grid:

Adjacency: (r, c) connects to:

[1, 1, 0]

* (r + 1, c) Down

[1, 0, 0]

* (r - 1, c) Up

[0, 0, 1]

* (r, c + 1) Right

Island 1: size 3

* (r, c - 1) Left

INTERVIEW CHALLENGE : COURSE SCHEDULE (LEETCODE 207)

QUESTION: Determine if you can finish all courses given prerequisite pairings (Cycle Detection in Directed Graph).

HINT: Topological Sort (Kahn's Algorithm via In-Degree Queue) or DFS with 3-color node visited states.

APPROACH & ANSWER: Compute in-degrees. Enqueue nodes with in-degree 0. While queue not empty, decrement neighbor in-degrees.

EXPLANATION: If total visited courses equals n, graph is a DAG (no cycle); otherwise cycle prevents graduation.

TIME: O(V + E) linear graph scan | SPACE: O(V + E) adjacency list

COMMON MISTAKE

Infinite recursion loop in graph DFS (forgotten visited set or in-place island sinking!).

PRO INTERVIEW TIP

For shortest path in unweighted graphs, BFS is guaranteed to reach target in minimum steps.

PART 12 : GRAPHS : CODE LAB

MULTI-LANGUAGE IMPLEMENTATIONS: **JAVA (Primary)** | C++ (STL) | **PYTHON (Clean)****JAVA : Number of Islands (LeetCode 200)**

```

public static int numIslands(char[][] grid) {
    if (grid == null || grid.length == 0) return 0;
    int count = 0;
    for (int r = 0; r < grid.length; r++) {
        for (int c = 0; c < grid[0].length; c++) {
            if (grid[r][c] == '1') { count++; dfs(grid, r, c); }
        }
    }
    return count;
}

private static void dfs(char[][] g, int r, int c) {
    if (r < 0 || r >= g.length || c < 0 || c >= g[0].length || g[r][c] != '1')
        return;
    g[r][c] = '0'; // Sink island in-place
    dfs(g, r+1, c); dfs(g, r-1, c); dfs(g, r, c+1); dfs(g, r, c-1);
}

```

* Mutating '1' to '0' eliminates visited set memory allocation.

C++ : Number of Islands (LeetCode 200)

```

void dfs(vector<vector<char>>& g, int r, int c) {
    if (r < 0 || r >= g.size() || c < 0 || c >= g[0].size() || g[r][c] != '1')
        return;
    g[r][c] = '0'; // Sink
    dfs(g, r+1, c); dfs(g, r-1, c); dfs(g, r, c+1); dfs(g, r, c-1);
}

int numIslands(vector<vector<char>>& grid) {
    int count = 0;
    for (int r = 0; r < grid.size(); r++) {
        for (int c = 0; c < grid[0].size(); c++) {
            if (grid[r][c] == '1') { count++; dfs(grid, r, c); }
        }
    }
    return count;
}

```

* Pass grid by reference vector<vector<char>>& to prevent copying the whole 2D grid.

PYTHON : Number of Islands (LeetCode 200)

```

def num_islands(grid: list[list[str]]) -> int:
    if not grid: return 0
    rows, cols, count = len(grid), len(grid[0]), 0
    def dfs(r: int, c: int):
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != '1': return
        grid[r][c] = '0' # Sink
        dfs(r+1, c); dfs(r-1, c); dfs(r, c+1); dfs(r, c-1)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1': count += 1; dfs(r, c)
    return count

```

* Nested helper function captures outer scope rows and cols cleanly.

SHARED COMPLEXITY: TIME $O(M * N)$ Total Grid Cells | SPACE $O(M * N)$ Worst Case DFS Stack

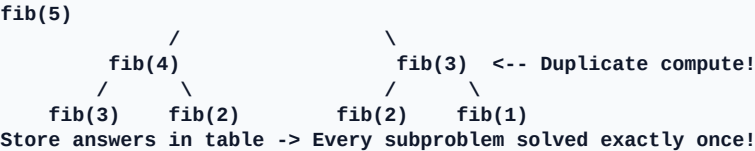
Key Takeaway: In-place sinking of visited nodes avoids allocating separate visited sets.

PART 13 : DYNAMIC PROGRAMMING

THE 5 CORE QUESTIONS ANSWERED

- 1. **WHAT:** Optimization technique breaking problem into overlapping subproblems, storing results.
- 2. **WHY:** Turns exponential brute-force $O(2^n)$ recursion into blazing fast linear $O(n)$ runtime.
- 3. **HOW:** 1. Top-Down (Memoization with Map). 2. Bottom-Up (Tabulation array). 3. Space optimization.
- 4. **WHEN:** Use when problem has overlapping subproblems AND optimal substructure property.
- 5. **RECOGNIZE:** Keywords: "maximum profit", "minimum cost", "ways to climb", "longest common subsequence".

OVERLAPPING SUBPROBLEMS IN FIBONACCI TREE



INTERVIEW CHALLENGE : COIN CHANGE (LEETCODE 322)

QUESTION: Find fewest number of coins needed to make up a given amount.

HINT: $dp[i]$ = minimum coins to make amount i . Initialize array with amount + 1.

APPROACH & ANSWER: $dp[0] = 0$. For $a = 1$ to amount: for each coin: if $(a \geq coin)$ $dp[a] = \min(dp[a], dp[a - coin] + 1)$;

EXPLANATION: Builds optimal minimum coin combination iteratively from 0 up to target amount.

TIME: $O(\text{amount} * \text{coins.length})$ | **SPACE:** $O(\text{amount})$ 1D dp table

COMMON MISTAKE

Greedy choice fails for arbitrary coin systems (e.g. amount 6 with coins [1, 3, 4] needs DP).

PRO INTERVIEW TIP

After writing a 1D $dp[n]$ solution, check if you only need the past 2 variables to get $O(1)$ space.

PART 13 : DYNAMIC PROGRAMMING : CODE LAB**MULTI-LANGUAGE IMPLEMENTATIONS:** **JAVA** (Primary) | **C++** (STL) | **PYTHON** (Clean)**JAVA : Climbing Stairs (LeetCode 70)**

```
public static int climbStairs(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

* Space optimized from $O(n)$ array to $O(1)$ by storing only past 2 subproblem states.

C++ : Climbing Stairs (LeetCode 70)

```
int climbStairs(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

* Optimal recurrence matches Fibonacci: $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$.

PYTHON : Climbing Stairs (LeetCode 70)

```
def climb_stairs(n: int) -> int:
    if n <= 2: return n
    prev2, prev1 = 1, 2
    for _ in range(3, n + 1):
        prev2, prev1 = prev1, prev1 + prev2
    return prev1
```

* Python tuple unpacking `prev2, prev1 = prev1, prev1 + prev2` updates simultaneously.

SHARED COMPLEXITY: TIME $O(n)$ Single Linear Pass | SPACE $O(1)$ Constant Variables

Key Takeaway: Identify recurrence relation, then optimize memory by retaining only necessary state.

PART 14 : 14 MASTER INTERVIEW PATTERNS

Recognize problem traits instantly and apply the proven master template:

1. Two Pointers

Sorted arrays, pairs with target sum, palindrome validation, in-place partitions.

2. Fast & Slow Pointers

Cycle detection in linked list/array, finding middle node, happy number loops.

3. Sliding Window

Contiguous subarray/substring problems with min, max, or target sum constraints.

4. Merge Intervals

Overlapping schedules, meeting room bookings, calendar conflict resolution.

5. Cyclic Sort

Arrays with numbers in fixed bounded range 1..n or 0..n (find missing/duplicate).

6. In-Place Reversal

Reversing sublists of a linked list without allocating new node objects.

7. Tree BFS

Level-order traversal, zigzag order, minimum depth, shortest path in tree.

8. Tree DFS

Root-to-leaf path sums, tree diameter, lowest common ancestor, subtree checks.

9. Two Heaps

Finding running median of a data stream, balancing lower and upper halves.

10. Subsets & Backtracking

Power sets, permutations, combinations, N-Queens, word search grid exploration.

11. Modified Binary Search

Rotated sorted arrays, boundary checks, search insert, peak element finding.

12. Top K Elements

Min-Heap of size k for kth largest; Max-Heap of size k for kth smallest.

13. K-Way Merge

Merging k sorted lists, streams, or matrices using a PriorityQueue.

14. 0/1 Knapsack & DP

Choosing items with weight and value constraints to maximize total value.

PART 15 : 90-DAY ROADMAP & PLAYBOOK

MY 90-DAY DSA ROADMAP

A structured journey from DSA Beginner -> Problem Solver -> Interview Ready

DAY 1 -> 30

Core Foundations: Big-O, Arrays, Strings, Two Pointers & 26-Bucket Array.

DAY 31 -> 60

Core Data Structures: Binary Search, Sorting, Linked Lists, Stacks, Queues & HashMaps.

DAY 61 [ACHIEVED]

MILESTONE 1: Complete DSA Foundation Completed!

DAY 62 -> 66 [IN PROGRESS]

Trees, BST, Heaps & Graphs (Actual Progress: 27 LeetCode Problems Solved!)

DAY 67 -> 75

Dynamic Programming, Monotonic Stacks & 14 Master Patterns (Target: 60 Problems).

DAY 76 -> 90

Mixed FAANG Company Sets, Live Timed Mocks & Edge-Case Drills (Target: 100 Problems).

90 DAYS COMPLETE * 100 LEETCODE PROBLEMS * DSA -> PATTERN RECOGNITION -> INTERVIEW READY

THE 6-STEP LIVE TECHNICAL INTERVIEW STRATEGY

Step 1 (1 min): Clarify Constraints

Ask about input size, negative numbers, duplicates, memory limits.

Step 2 (2 min): State Brute Force

Propose obvious $O(n^2)$ solution first. Establishes baseline confidence!

Step 3 (2 min): State the Pattern

Explain why a specific structure (e.g. Two Pointers, HashMap) drops time.

Step 4 (2 min): Dry Run on Example

Trace indices and pointers on scratchpad before touching the keyboard.

Step 5 (12 min): Write Clean Code

Use meaningful variable names, modular helpers, and handle edge cases.

Step 6 (3 min): Verify Edge Cases

Test empty input, single element, negative numbers, and all duplicates.

A PERSONAL CLOSING NOTE FROM SOWMYA

DSA is not a test of raw IQ; it is purely a skill of pattern recognition and deliberate practice.

Do not just memorize solutions. Trace pointers on paper, understand the underlying trade-offs, and celebrate every concept you conquer.

You have everything it takes to succeed in your coding journey. Keep coding and keep moving forward!